

Solution and Explanation - Implement Stack Class

Explanation

public class Stack { : This begins the definition of a **Stack** class. The class is **public**, which means it can be accessed from any other class in the same package or outside of the package.

public class Node { : Inside the **Stack** class, we define another class called **Node**. This inner class is **public**, allowing it to be accessed from outside the **Stack** class.

int value; : Inside the **Node** class, an instance variable named **value** of type **int** is declared. This variable will hold the integer value of the node.

Node next; : A **Node** type instance variable named **next** is declared. This will store the reference/link to the next node in the stack. If there's no subsequent node, it will be **null**.

Node(int value) { : A constructor for the **Node** class is defined. It takes an integer **value** as its parameter, which will be assigned to the **value** instance variable when a new **Node** instance is created.

this.value = value; : The passed-in **value** to the constructor is assigned to the **value** instance variable of the **Node** object. The **this** keyword differentiates the instance variable **value** from the constructor parameter **value**.

this.next = null; : The **next** instance variable of the **Node** is initialized to **null**, meaning by default, the node doesn't point to any other node.

public Node top; : In the **Stack** class, a public instance variable named **top** of type **Node** is declared. This variable will reference the topmost node in the stack.

public int height; : Another public instance variable named **height** of type **int** is declared in the **Stack** class. This variable will keep track of the number of elements/nodes in the stack.

public Stack(int initialValue) { : This is the constructor of the **Stack** class. It's designed to be called when a new **Stack** object is created. It takes an

integer **initialValue** as its parameter, which represents the value of the first node to be added to the stack.

this.top = new Node(initialValue); : Inside the constructor, a new **Node** object with the provided **initialValue** is created. This new node becomes the topmost node of the stack.

this.height = 1; : As one node has been added to the stack upon its initialization, the **height** instance variable is set to 1, indicating the current size of the stack.

Time and Space Complexity

1. Node Class:

- **Attributes (int value & Node next):**
 - **Space:** Both the **value** and **next** attributes use constant space, so their space complexity is **O(1)**.
- **Constructor (Node(int value)):**
 - **Time:** Assigning the **value** and setting **next** to null both happen in constant time. Thus, the time complexity is **O(1)**.
 - **Space:** No additional space is being utilized other than the already allocated node itself. So, the space complexity remains **O(1)**.

2. Stack Class:

- **Attributes (Node top & int height):**
 - **Space:** Both the **top** and **height** attributes use constant space, so their space complexity is **O(1)**.
- **Constructor (Stack(int initialValue)):**
 - **Time:** Creating a new **Node** and assigning to the **top** and initializing the **height** are all operations that complete in constant time. Hence, the time complexity is **O(1)**.
 - **Space:** The creation of a new **Node** instance requires constant space. Other attributes (**top** and **height**) are also using constant space. Consequently, the space complexity is **O(1)**.

For the provided **Stack** class code:

- All operations (attribute assignments and constructors) are done in constant time, so the time complexity is **O(1)** for each operation.
- All variables and objects being utilized (node attributes, top of stack, height) are using constant space, hence the space complexity for them is **O(1)**.

Solution and Explanation - Push Method

Let's break down the **push** method of your **Stack** class, line by line, to understand its functionality. This method is designed to add a new element to the top of the stack.

```
public void push(int value) {
```

- This line defines the **push** method. It's a public method, meaning it can be accessed from outside the **Stack** class. The method takes one argument, **value**, which is an integer that represents the data to be added to the stack.
`Node addedNode = new Node(value);`

- A new **Node** object, **addedNode**, is created with the given **value**. The **Node** constructor sets this **value** to the **value** field of the **Node** and initializes **next** to **null**.

```
if (length == 0) {
```

- This **if** statement checks if the stack is empty. The stack is considered empty when **length**, which tracks the number of elements in the stack, is **0**.
`peak = addedNode;`

- If the stack is empty (**length == 0**), this line sets the **peak** (the top element of the stack) to be the **addedNode**. This means the added node is now the only element in the stack.

```
} else {
```

- This **else** block is executed if the stack is not empty (**length != 0**).

```
addedNode.next = peak;
```

- In the **else** block, the **next** of the **addedNode** is set to the current **peak**. This means that the **addedNode** now points to the previous top element of the stack.

```
peak = addedNode;
```

- Still in the **else** block, this line updates the **peak** to be the **addedNode**. Now, **addedNode** is the new top element of the stack.

```
}
```

- This closes the **if-else** statement.

```
length++; }
```

- Finally, the **length** of the stack is incremented by 1, reflecting the addition of the new element. The method then concludes.

Overall, the **push** method adds a new element to the top of the stack, adjusts the **peak** to reference this new node, and increments the stack's **length**.

Solution/Explanation and Time/Space Complexity - Pop Method

```
public class Stack {  
    private Node top;  
    public int height;  
  
    public Stack() {  
        top = null;  
        height = 0;  
    }  
  
    public Node pop() {  
        if (height == 0) {  
            return null;  
        }  
        Node temp = top;  
        top = top.next;  
        temp.next = null;  
        height--;  
        return temp;  
    }  
  
    public void push(Node node) {  
        node.next = top;  
        top = node;  
        height++;  
    }  
}
```

Let's break down the **pop** method in your **Stack** class line by line and then discuss its time and space complexity:

```
public Node pop() { if (height == 0) { return null; }  
Node temp = top;
```

- This checks if the stack is empty (height is 0). If so, it returns **null**, indicating there's nothing to pop.

```
Node temp = top;
```

- This line creates a temporary **Node** object, **temp**, and assigns it the value of **top**. This effectively stores the current top node of the stack.

```
top = top.next;
```

- Here, the **top** pointer of the stack is updated to point to the next node in the stack. This effectively removes the original top node from the stack.

```
temp.next = null;
```

- This line disconnects the popped node from the stack by setting its **next** pointer to **null**.

```
height--;
```

- The stack's height (or size) is decremented by 1, reflecting the removal of one node.

```
return temp; }
```

- Finally, the method returns the **temp** node, which is the node that was just removed from the stack.

Time Complexity:

- The **pop** method has a time complexity of $O(1)$, or constant time. This is because the method performs a fixed number of operations regardless of the size of the stack.

Space Complexity:

- The space complexity of the **pop** method is also $O(1)$. The method only creates a single temporary variable (**temp**) and doesn't use any additional data structures that grow with the size of the input.

In summary, the **pop** method is efficient in both time and space, adhering to the standard characteristics of stack operations in terms of complexity.

Solution/Explanation and Time/Space Complexity of Balanced Parentheses Checker

```
import java.util.Stack;
```

```
public class Exercise {  
    public static boolean isBalanced(String expression) {  
        Stack<Character> stack = new Stack<>();  
        for (int i = 0; i < expression.length(); i++) {  
            char c = expression.charAt(i);  
            if (c == '(' || c == '[' || c == '{') {  
                stack.push(c);  
            } else if (c == ')' || c == ']' || c == '}') {  
                if (stack.isEmpty()) {  
                    return false;  
                }  
                char top = stack.pop();  
                if ((c == ')' && top != '(') || (c == ']' && top != '[')  
&& top != '{') {  
                    return false;  
                }  
            }  
        }  
    }  
}
```

```

    }
}
return stack.isEmpty();
}
}

```

Explanation.

1. `import java.util.Stack;`
 - This line imports the `Stack` class from the `java.util` package. The `Stack` class represents a last-in-first-out (LIFO) stack of objects.
2. `public class Exercise {`
 - This line defines a new class named `Exercise`.
3. `public static boolean isBalanced(String expression) {`
 - This line starts the definition of a `static` method named `isBalanced`, which returns a boolean value (`true` or `false`). This method takes one argument, a `String` named `expression`, which represents the expression to be checked for balanced parentheses.
4. `Stack<Character> stack = new Stack<>();`
 - A `Stack` of `Character` objects is created and named `stack`. This stack will be used to keep track of opening parentheses in the `expression`.
5. `for (int i = 0; i < expression.length(); i++) {`
 - This line marks the start of a `for` loop that iterates over each character in `expression`. The loop continues as long as `i` is less than the length of `expression`.
6. `char c = expression.charAt(i);`
 - Inside the loop, each character of the `expression` is accessed one by one and stored in the variable `c`.
7. `if (c == '(' || c == '[' || c == '{') {`
 - This `if` statement checks if the current character `c` is an opening parenthesis – either `(`, `[`, or `{`.
8. `stack.push(c);`
 - If `c` is an opening parenthesis, it is pushed onto the stack. This is used to keep track of the most recent unmatched opening parenthesis.
9. `} else if (c == ')' || c == ']' || c == '}') {`
 - This `else if` statement checks if `c` is a closing parenthesis – either `)`, `]`, or `}`.
10. `if (stack.isEmpty()) {`
 - Inside the `else if`, this line checks if the stack is empty when a closing parenthesis is encountered. If it's empty, there's no corresponding opening parenthesis, so the method returns `false`.

11. `char top = stack.pop();`
 - This line pops the top element from the stack. This element represents the most recent unmatched opening parenthesis.
12. `if ((c == ')') && top != '(') || (c == ']') && top != '[') || (c == '}') && top != '{')) {`
 - This `if` statement checks if the current closing parenthesis `c` matches the popped opening parenthesis `top`. If they don't match, it returns `false`.
13. `return false;`
 - This line returns `false` if either there's no corresponding opening parenthesis or if there's a mismatch in the types of parentheses.
14. `}`
 - End of the `for` loop and `if-else` blocks.
15. `return stack.isEmpty();`
 - After the entire string is processed, this line returns `true` if the stack is empty (meaning all opening parentheses were matched with closing ones), or `false` if there are still unmatched opening parentheses in the stack.
16. `}`
 - End of the `isBalanced` method.
17. `}`
 - End of the `Exercise` class.

Time and Space Complexity

1. `import java.util.Stack;`
 - **Complexity:** No impact. This is an import statement and does not affect the runtime or space complexity of the algorithm.
2. `public class Exercise { ... }`
 - **Complexity:** No impact. Class definition does not contribute to runtime or space complexity.
3. `public static boolean isBalanced(String expression) { ... }`
 - **Complexity:** No impact. Method definition itself does not contribute to runtime or space complexity.
4. `Stack<Character> stack = new Stack<>();`
 - **Space Complexity:** The stack is where space complexity comes into play. In the worst case (all characters are opening brackets), the space complexity is $O(n)$, where n is the length of the string.
5. `for (int i = 0; i < expression.length(); i++) {`

- **Time Complexity:** This for loop is the main contributor to the time complexity. It runs once for each character in the input string, leading to $O(n)$ time complexity.
6. `char c = expression.charAt(i);`
 - **Time Complexity:** $O(1)$ for each iteration. This operation is constant time but occurs within the $O(n)$ loop.
 - **Space Complexity:** $O(1)$, just a single character storage.
 7. `if (c == '(' || c == '[' || c == '{') { ... }`
 - **Time Complexity:** $O(1)$ for each check. It's a constant-time operation within the $O(n)$ loop.
 8. `stack.push(c);`
 - **Time Complexity:** $O(1)$ for each push operation.
 - **Space Complexity:** Contributes to the worst-case space complexity of $O(n)$ as discussed in point 4.
 9. `} else if (c == ')' || c == ']' || c == '}') { ... }`
 - **Time Complexity:** $O(1)$ for each check, within the $O(n)$ loop.
 10. `if (stack.isEmpty()) { ... }`
 - **Time Complexity:** $O(1)$ for each check. Checking if a stack is empty is a constant-time operation.
 11. `char top = stack.pop();`
 - **Time Complexity:** $O(1)$ for each pop operation.
 - **Space Complexity:** Does not significantly affect space complexity, as it's removing elements from the stack.
 12. `if ((c == ')') && top != '(' || (c == ']') && top != '[' || (c == '}') && top != '{')) { ... }`
 - **Time Complexity:** $O(1)$ for each check. It's a constant-time comparison.
 13. `return false;`
 - **Time Complexity:** $O(1)$. Returning a value is a constant-time operation.
 14. `return stack.isEmpty();`
 - **Time Complexity:** $O(1)$. Checking if the stack is empty is a constant-time operation.

Overall:

- **Time Complexity:** $O(n)$, where n is the length of the string. This is due to the single traversal of the string.
- **Space Complexity:** $O(n)$ in the worst case, where all characters are opening brackets and thus pushed onto the stack. In the average or best case, the space complexity will be less.